Guía completa — Sistema de Gestión Universitaria

Guía general + paso a paso + commits de Git

Objetivo. Construir el proyecto vos mismo, usando esta guía como mapa. Cada etapa debe quedar probada antes de pasar a la siguiente.

Stack: HTML5, CSS3, JavaScript, Node.js, Express, PostgreSQL, pgAdmin, Git, GitHub, Vercel y Azure App Service. PostgreSQL será local durante el desarrollo y se migrará a Azure al final. Python queda como módulo opcional posterior.

Arquitectura final

```
GitHub → Vercel (frontend)
GitHub → Azure App Service (Node.js + Express)
Azure App Service → Azure PostgreSQL
```

Durante el desarrollo

VS Code → Frontend + Node/Express + PostgreSQL local. No necesitamos gastar créditos de Azure mientras construimos y probamos el sistema.

1. Preparación

Definir requisitos, instalar herramientas, crear GitHub y establecer convenciones.

2. Proyecto base

Crear Node, Express, estructura de carpetas, .gitignore, variables de entorno y servidor.

3. Base de datos

Diseñar PostgreSQL con usuarios, roles, estudiantes, profesores, programas, materias, matrículas, horarios y calificaciones.

4. Backend

Conectar PostgreSQL, organizar routes/controllers/services/repositories y crear manejo de errores.

5. Seguridad

Login, hash de contraseñas, autenticación y autorización por roles.

6. Módulos

Implementar los diez requisitos: estudiantes, profesores, materias, programas, matrículas, horarios, calificaciones, historial, reportes y roles.

7. Frontend

Login, navegación por rol, formularios, tablas, dashboards sencillos y consumo de API.

8. Pruebas

Probar API, permisos, validaciones, integración y flujo completo.

9. GitHub/producción

Limpiar secretos, actualizar README y crear versión estable.

10. Nube

Vercel para frontend; Azure PostgreSQL y Azure App Service para producción.

11. Python opcional

Solo después del sistema principal y si aporta valor sin aumentar demasiado el riesgo.

12. Sustentación

Diagrama, explicación de BD/API/seguridad, demostración y plan alternativo ante fallos de Internet.

Paso a paso y commits

Paso 0 — Preparar el PC

• Instalar VS Code, Node.js LTS, PostgreSQL, pgAdmin y Git.

• Comprobar versiones con node --version, npm --version y git --version.

• Comprobar que PostgreSQL funciona desde pgAdmin.

Commit sugerido: `chore: preparar entorno de desarrollo`

Paso 1 — GitHub y Node

• Crear repositorio en GitHub y clonarlo.

• Ejecutar npm init -y.

• Crear README.md.

• Crear .gitignore para node_modules y .env.

• Crear .env.example sin secretos.

Commit sugerido: `chore: inicializar proyecto y control de versiones`

Paso 2 — Express

• Instalar express y nodemon.

• Crear el servidor.

• Crear GET /api/health.

• Ejecutar y comprobar la respuesta JSON.

Commit sugerido: `feat: crear servidor express base`

Paso 3 — Arquitectura

• Crear carpetas config, routes, controllers, services, repositories y middlewares.

• Separar app y server si resulta conveniente.

• Crear manejo centralizado de errores.

Commit sugerido: `refactor: organizar arquitectura del backend`

Paso 4 — PostgreSQL local

• Crear la base de datos en pgAdmin.

• Crear schema.sql.

• Definir PK, FK, UNIQUE, NOT NULL y CHECK.

• Crear las tablas principales.

Commit sugerido: `feat: crear esquema inicial de base de datos`

Paso 5 — Conexión BD

• Instalar el driver de PostgreSQL.

• Crear un pool usando variables de entorno.

• Probar SELECT 1 desde Node.

• Verificar conexión antes de continuar.

Commit sugerido: `feat: conectar backend con postgresql`

Paso 6 — Usuarios y roles

• Crear usuarios y roles.

• Implementar hash de contraseñas.

• Crear login.

• Crear middleware de autenticación.

• Crear autorización para administrador, profesor y estudiante.

Commit sugerido: `feat: implementar autenticacion y roles`

Paso 7 — Estudiantes y profesores

• Crear tablas, rutas, controladores y consultas.

• Implementar CRUD.

• Agregar validaciones.

• Probar endpoints.

Commit sugerido: `feat: agregar gestion de estudiantes y profesores`

Paso 8 — Programas y materias

• Crear programas académicos.

• Crear materias.

• Relacionar materias y programas.

• Implementar CRUD y permisos.

Commit sugerido: `feat: agregar programas y materias`

Paso 9 — Matrículas

• Relacionar estudiante con la oferta/materia definida en el modelo.

• Evitar duplicados.

• Validar permisos.

• Probar creación y consulta.

Commit sugerido: `feat: implementar matriculas`

Paso 10 — Horarios

• Crear horarios.

• Implementar CRUD.

• Evitar conflictos de horario.

• Permitir consultas según rol.

Commit sugerido: `feat: implementar gestion de horarios`

Paso 11 — Calificaciones

• Crear calificaciones vinculadas a la matrícula.

• Permitir modificación solo a roles autorizados.

• Crear consulta de historial académico.

• Probar cálculos y consultas.

Commit sugerido: `feat: implementar calificaciones e historial`

Paso 12 — Reportes

• Crear endpoints de reportes.

• Agregar filtros necesarios.

• Respetar permisos.

• Probar datos reales de demostración.

Commit sugerido: `feat: agregar reportes academicos`

Paso 13 — Frontend base

• Crear estructura HTML/CSS/JS.

• Crear login.

• Conectar login con la API.

• Mostrar navegación según rol.

Commit sugerido: `feat: crear frontend base y autenticacion`

Paso 14 — Frontend de módulos

• Crear pantallas de estudiantes, profesores, materias, programas, matrículas, horarios, calificaciones e historial.

• Consumir API con fetch.

• Mostrar carga, errores y confirmaciones.

Commit sugerido: `feat: implementar modulos del frontend`

Paso 15 — Seguridad e integración

• Revisar CORS.

• Probar que un rol no pueda ejecutar funciones de otro.

• Revisar validación backend.

• Eliminar credenciales del código.

Commit sugerido: `fix: reforzar seguridad e integracion`

Paso 16 — Pruebas finales

• Probar los tres roles.

• Probar casos inválidos.

• Probar flujo estudiante → matrícula → horario → calificación → historial.

• Preparar datos de demostración.

Commit sugerido: `test: completar pruebas de integracion`

Paso 17 — Preparar producción

• Revisar package.json y scripts.

• Revisar .env y .env.example.

• Actualizar README.

• Crear versión estable.

Commit sugerido: `chore: preparar version de produccion`

Paso 18 — Frontend en Vercel

• Conectar GitHub con Vercel.

• Configurar la URL del backend mediante variable de entorno.

• Publicar y probar.

Commit sugerido: `deploy: publicar frontend en vercel`

Paso 19 — PostgreSQL en Azure

• Crear PostgreSQL en Azure cuando llegue la fase final.

• Migrar esquema y datos de demostración.

• Configurar acceso seguro.

• Probar conexión.

Commit sugerido: `deploy: migrar base de datos a azure`

Paso 20 — Backend en Azure

• Crear Azure App Service para Node.js.

• Configurar variables de entorno.

• Desplegar desde GitHub/VS Code.

• Revisar logs.

• Probar /api/health.

• Conectar con PostgreSQL Azure.

Commit sugerido: `deploy: publicar backend en azure`

Paso 21 — Integración final

• Actualizar la URL de API en Vercel.

• Configurar CORS para el dominio de Vercel.

• Probar login, roles y módulos completos.

• Guardar evidencias para la sustentación.

Commit sugerido: `fix: completar integracion de produccion`

Paso 22 — Entrega

• Crear tag/release final.

- Actualizar README.
- Preparar diagrama de arquitectura.
- Ensayar la sustentación.
- Tener datos y plan alternativo de demostración.

Commit sugerido: `docs: preparar entrega y sustentacion`

Cómo trabajar con Git

• Antes de comenzar: git status.

• Trabajar en una funcionalidad concreta.

• Probar la aplicación antes del commit.

• Revisar git diff.

• Crear el commit solo cuando el bloque esté estable.

• Hacer git push.

• No subir .env, contraseñas, tokens ni claves.

• Usar mensajes claros: feat (funcionalidad), fix (corrección), refactor (estructura), test (pruebas), docs (documentación), chore (mantenimiento), deploy (despliegue).

Checklist de sustentación

■ Login funciona.

■ Roles y permisos funcionan.

■ Estudiantes funcionan.

■ Profesores funcionan.

■ Programas y materias funcionan.

■ Matrículas funcionan y evitan duplicados.

■ Horarios funcionan.

■ Calificaciones funcionan.

■ Historial académico funciona.

■ Reportes funcionan.

■ Frontend publicado en Vercel.

■ Backend respondiendo desde Azure.

■ PostgreSQL de producción funcionando.

■ Variables de entorno configuradas.

■ CORS configurado.

■ GitHub no contiene secretos.

■ README actualizado.

■ Flujo completo probado de principio a fin.

Nota: los precios, límites y planes de los proveedores cloud pueden cambiar. Para el despliegue final se revisarán las condiciones vigentes y el consumo antes de crear recursos de pago.